

Carona UCB

25 de Setembro de 2026



Visão geral

Um grupo de estudantes de uma universidade pretende lançar o CaronaUCB, um aplicativo de caronas solidárias entre alunos, professores e servidores de um mesmo campus universitário. A ideia surgiu após uma pesquisa interna apontar que mais de 60% dos estudantes gastam, em média, 40 minutos por trajeto usando transporte público, e que muitos já compartilham caronas informalmente por grupos de WhatsApp, sem qualquer controle de segurança, rota ou confirmação. Os empreendedores desejam um aplicativo em que motoristas cadastrados possam oferecer vagas em seus trajetos até o campus, e passageiros possam buscar, solicitar e confirmar caronas compatíveis com seu horário e localização. A universidade exige que apenas pessoas vinculadas à instituição (com e-mail institucional) possam se cadastrar, e que o sistema registre avaliações mútuas entre motorista e passageiro após cada corrida, por questões de segurança. A diretoria de TI da universidade também exige que os dados de localização dos usuários sejam mantidos por, no máximo, 90 dias, e que o aplicativo estivesse disponível tanto em Android quanto iOS, com tempo de resposta abaixo de 2 segundos nas buscas por carona.



Produto

Nome:

Carona UCB

Problema:

Alta dependência de transporte público e uso de meios informais para compartilhamento de caronas sem controle de segurança, registro de rotas ou garantia de vínculo institucional.

Usuário principal:

Estudantes, professores e demais funcionários da Universidade.

Sprint goal:

Validar o fluxo principal do CaronaUCB, permitindo que a comunidade acadêmica consiga se cadastrar, oferecer, buscar e solicitar caronas com segurança.

Backlog

User Story 1:

“Como estudante, quero me cadastrar no app usando meu e-mail institucional, para garantir que apenas pessoas da UCB possam acessar o Carona UCB.”

Prioridade:

Alta (Requisito fundamental de segurança exigido pela universidade).

Pontos:

5

Critérios de Aceitação:

1. O sistema deve bloquear cadastros de domínios que não sejam da universidade.
2. Um link de validação deve ser enviado para o e-mail cadastrado.

User Story 2:

“Como motorista, quero cadastrar minha rota e o número de vagas, para oferecer carona a outras pessoas da instituição.”

Prioridade:

Alta (Gera a oferta do serviço, essencial para o app existir).

Pontos:

8

Critérios de Aceitação:

1. Formulário deve conter origem, destino, data, horário e quantidade de assentos.
2. A carona deve aparecer como "disponível" no sistema imediatamente após ser salva.

User Story 3:

“Como passageiro, quero buscar caronas disponíveis filtrando por horário e proximidade, para encontrar a viagem que melhor se adapta à minha necessidade.”

Prioridade:

Alta (Necessário para a conexão entre o passageiro e o motorista).

Pontos:

8

Critérios de Aceitação:

1. O tempo de resposta da requisição de busca deve ser estritamente inferior a 2 segundos em ambas as plataformas (Android e iOS).
2. O resultado deve omitir caronas que já estejam lotadas.

User Story 4:

“Como passageiro, quero solicitar uma vaga em uma carona encontrada, para garantir meu lugar no trajeto.”

Prioridade:

Alta (Estabelece a conexão entre as duas partes).

Pontos:

5

Critérios de Aceitação:

1. O motorista recebe a notificação da solicitação.
2. A vaga solicitada fica com status "pendente" até a aprovação.

User Story 5:

“Como usuário, quero avaliar o motorista ou passageiro com quem viajei, para criar um histórico de confiabilidade na plataforma.”

Prioridade:

Média (Crucial para retenção e segurança contínua, mas não impede o primeiro teste de uso).

Pontos:

5

Critérios de Aceitação:

1. O sistema deve liberar a tela de avaliação (1 a 5 estrelas) assim que o horário previsto da carona for atingido.
2. Avaliações abaixo de 2 estrelas devem abrir um campo de texto obrigatório para justificativa.

User Story 6:

“Como gerente de TI, quero que os dados de localização dos usuários sejam apagados após 90 dias, para cumprir com as normativas de privacidade da universidade e com a regulamentação legal.”

Prioridade:

Média (Requisito obrigatório, mas o prazo de 90 dias permite que seja entregue em Sprints futuras).

Pontos:

3

Critérios de Aceitação:

1. Um código deve rodar diariamente limpando registros maiores que 90 dias.
2. O código deve registrar a exclusão em um arquivo.

User Story 7:

“Como motorista, quero aceitar ou recusar as solicitações recebidas, para ter controle total sobre quem entra no meu veículo.”

Prioridade:

Média (Poderia ser integrado ao fluxo da US4, mas foi separado por envolver UI do motorista).

Pontos:

3

Critérios de Aceitação:

1. Ao aceitar, o número de vagas da carona diminui em 1.
2. O passageiro é notificado da resposta.

User Story 8:

“Como usuário parte de uma carona, quero trocar mensagens pelo próprio app, para combinar o ponto de encontro sem divulgar meu WhatsApp ou outras redes sociais.”

Prioridade:

Baixa (Os usuários podem se encontrar em pontos pré-definidos do campus nesta fase inicial).

Pontos:

8

Critérios de Aceitação:

1. O chat só é ativado após o status da carona mudar para "Aceita".
2. O histórico de mensagens é apagado 24 horas após o fim da viagem.

Sprint Backlog

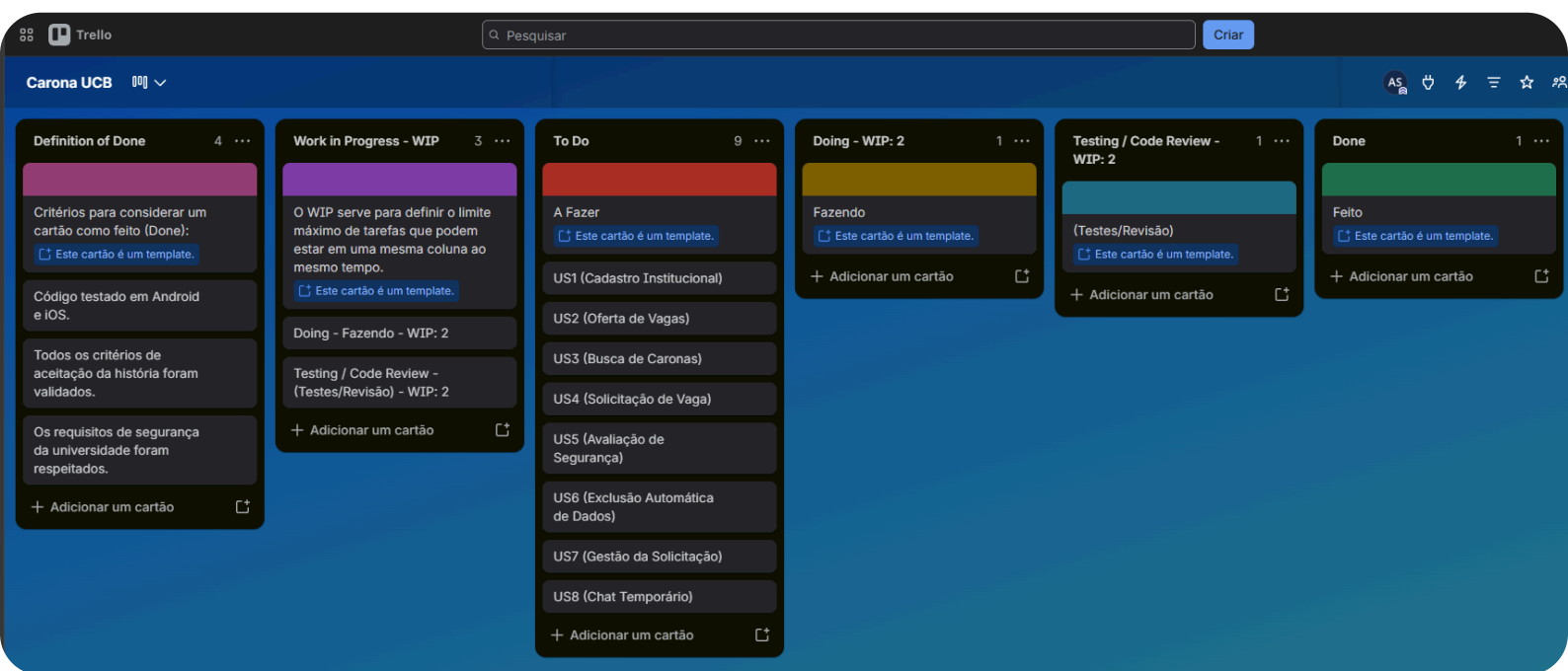
Para a Sprint inicial, as histórias escolhidas foram a US1 (5 pts), a US2 (8 pts), a US3 (8 pts) e a US4 (5 pts), totalizando 26 Story Points. A priorização por essas User Stories se baseia no fato de que essas quatro histórias integram o fluxo principal do app e contribuem diretamente para alcançar o Sprint Goal.

Com a resolução das demandas propostas pelas User Stories citadas acima, o app Carona Ucb já consegue ganhar forma e funcionar de forma satisfatória, uma vez que o usuário já conseguirá se cadastrar com a devida validação, bem como criar uma oferta, executar uma busca com tempo inferior a 2 segundos e solicitar uma vaga. As demais histórias, nomeadamente a US5, a US6, a US7 e a US8, não foram incluídas nesta etapa pois dependem do funcionamento prévio do fluxo de caronas ou possuem prazos mais extensos, de modo que sua inclusão excederia a capacidade da equipe para esta Sprint e poderia comprometer a qualidade da entrega inicial do produto.

Kanban, WIP e Definition of Done

Para gerenciar o fluxo de trabalho da nossa Sprint simulada, utilizamos o Trello como o nosso quadro Kanban.

A configuração inicial do quadro ficou da seguinte maneira:



Configuração inicial - Print Trello

Link para o quadro no Trello: [Clique Aqui](#)

O quadro apresenta as colunas clássicas do Kanban, sendo o *To Do* (A Fazer), onde ficam as histórias que entraram no Sprint Backlog, mas ainda não começaram; o *Doing* (Fazendo), que concentra os cartões em que a equipe está trabalhando no momento;

Testing / Code Review (Testes/Revisão), etapa para garantir a qualidade antes da entrega; e o *Done* (Feito), para onde vão os cartões quando estão 100% concluídos e devidamente testados.

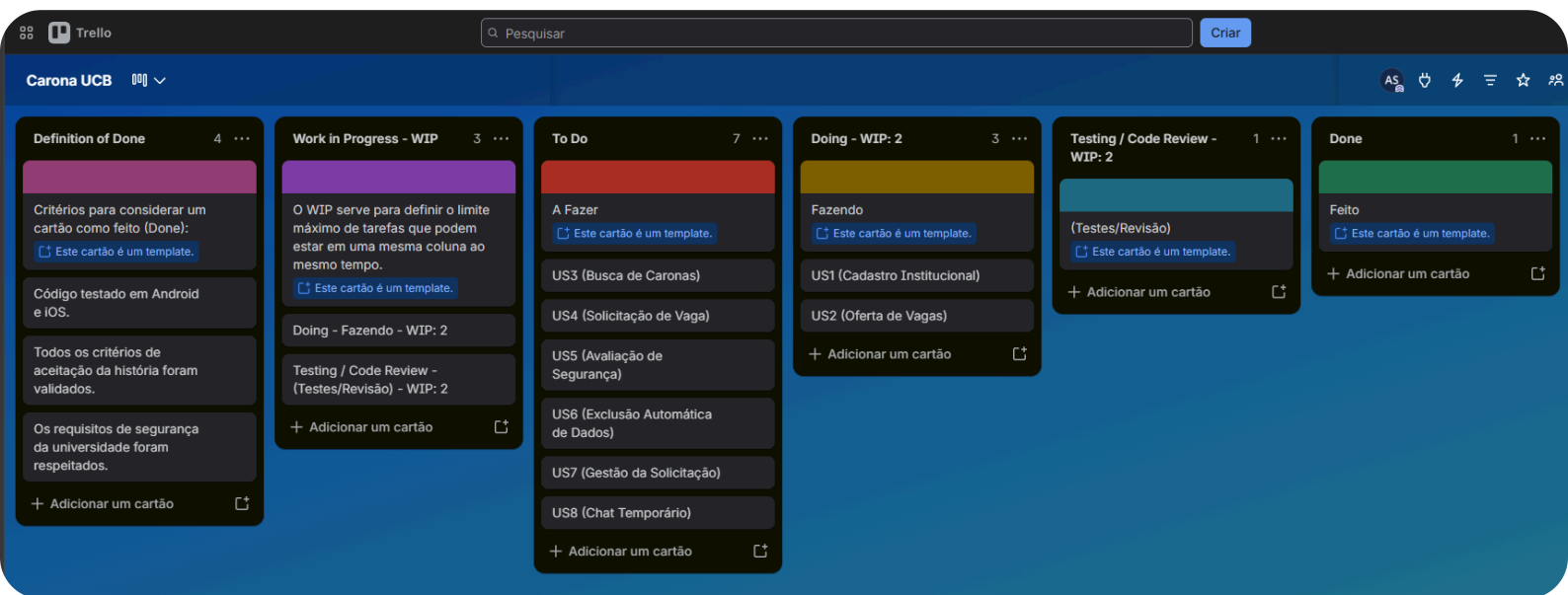
Já o limite de trabalho em progresso (WIP) foi definido em, no máximo, **duas tarefas** simultâneas para as colunas *Doing* e *Testing / Code Review*, com o intuito de impedir que a equipe inicie diversas atividades ao mesmo tempo, evitando gargalos e a perda de foco no projeto.

Quanto ao *Definition of Done* (Definição de Pronto), estabelecemos que um cartão só pode ser movido para a coluna final se o código tiver sido testado em Android e iOS, os critérios de aceitação da história foram validados e os requisitos de segurança da universidade foram rigorosamente respeitados.

Simulação da Sprint

Conforme já citado anteriormente, para a simulação da sprint decidimos puxar apenas as histórias **US1, US2, US3 e US4** para compor o Sprint Backlog. As histórias **US5, US6, US7 e US8** foram mantidas no *To Do*, pois excederiam a capacidade produtiva da equipa e não impedem a validação do fluxo principal do CaronaUCB, que é o nosso Sprint Goal. Cabe lembrar também que as User Stories já estão organizadas na coluna *To Do* por ordem de prioridade.

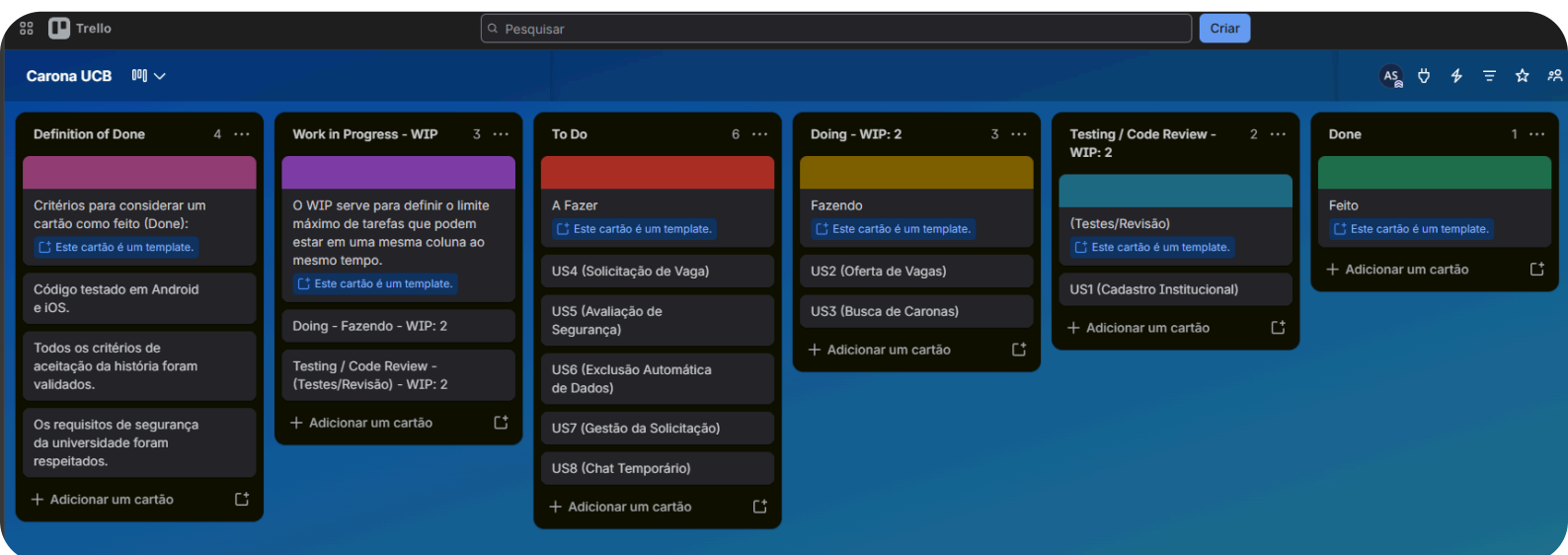
Dias 1 a 3:



Dias 1 a 3 da Sprint - Print Trello

Iniciamos a Sprint movendo a US1 (Cadastro Institucional) e a US2 (Oferta de Vagas) da coluna To Do para a coluna Doing, assim por atingir o limite de trabalho (WIP-2), nenhum outro cartão foi puxado.

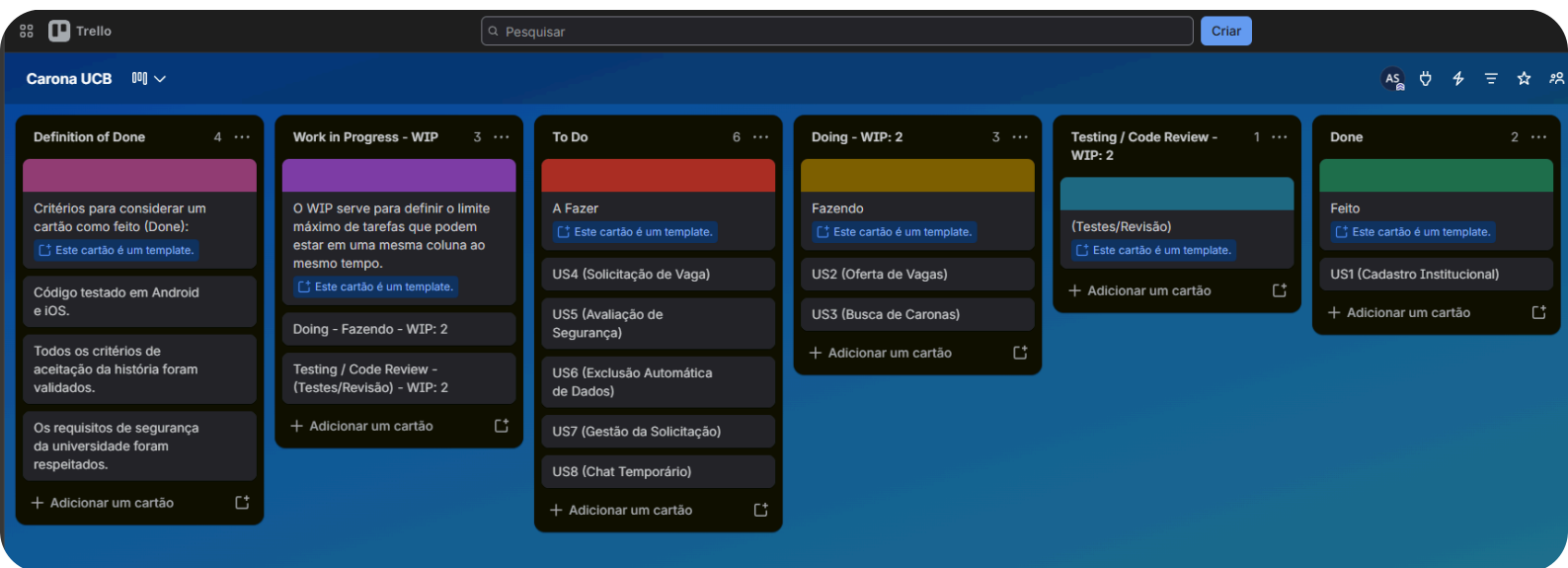
Dia 4:



Dia 4 da Sprint - Print Trello

A US1 foi finalizada no desenvolvimento e passou para a lista Testing / Code Review, desse modo a coluna Doing ficou com um espaço vazio, permitindo que a US3 fosse puxada para a coluna.

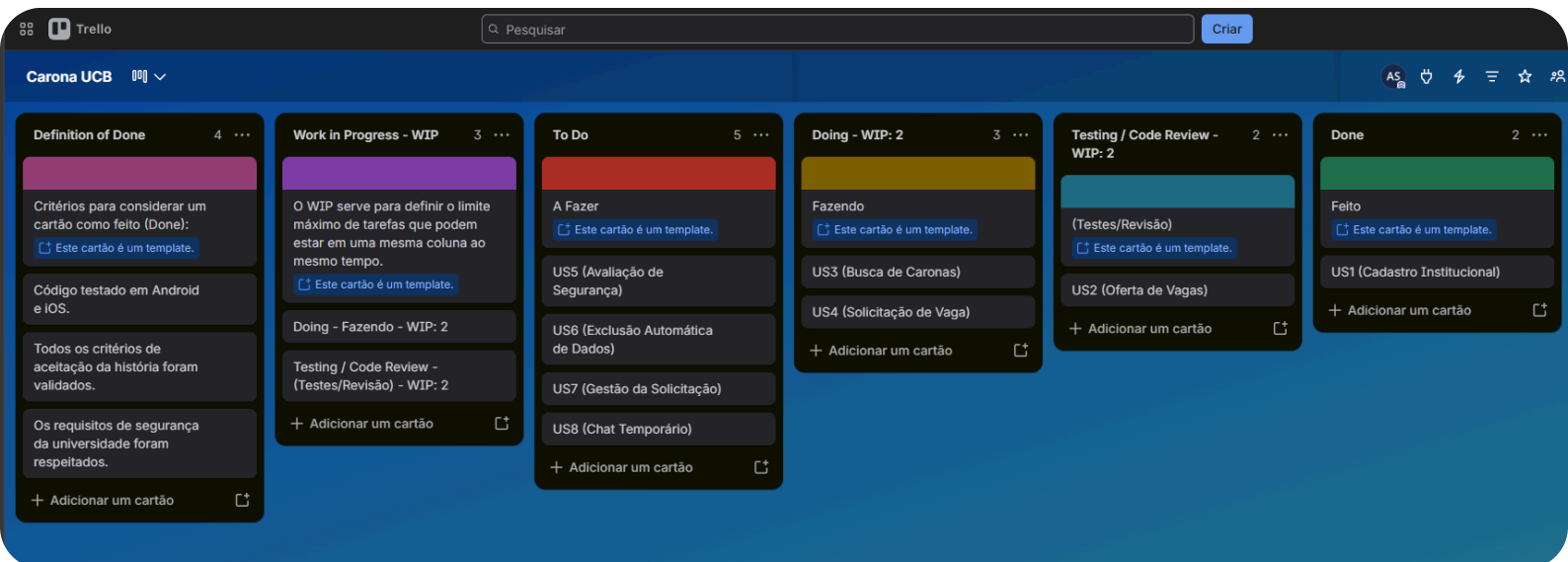
Dia 5:



Dia 5 da Sprint - Print Trello

A US1 foi testada seguindo todos os critérios estabelecidos no Definition of Done, e foi movida para a coluna Done.

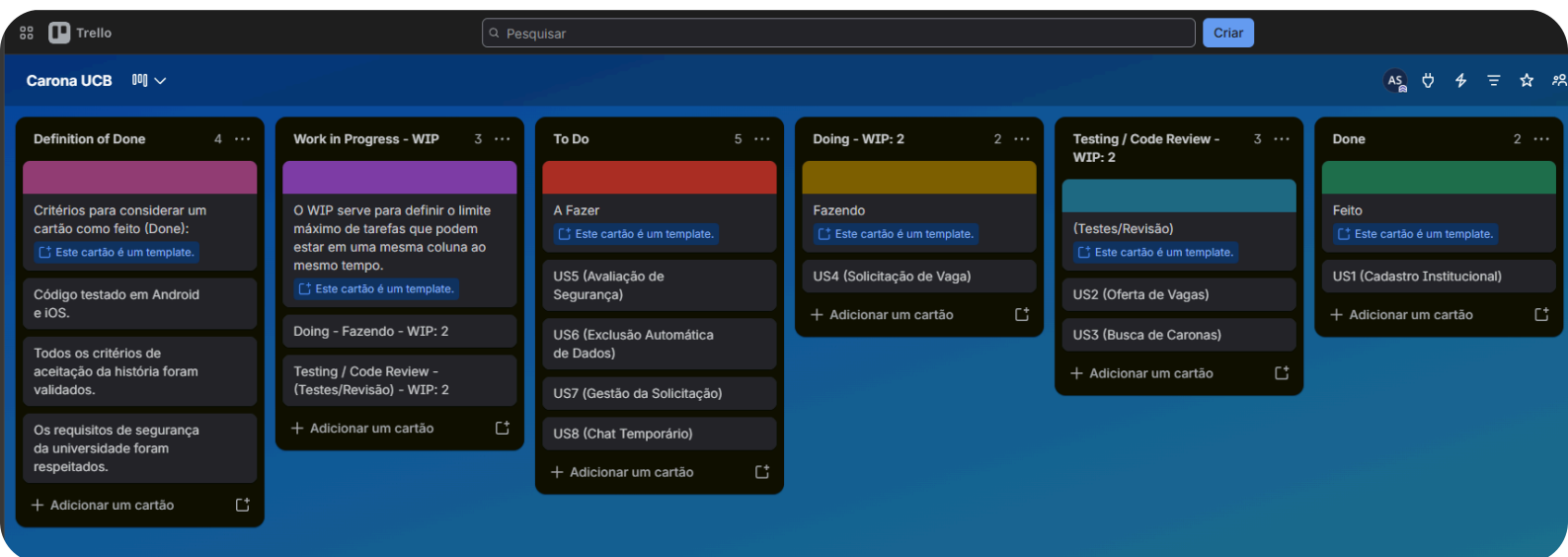
Dias 6 a 10:



Dias 6 a 10 da Sprint - Print Trello

A US2 foi movida para Testing / Code Review. Assim, a US4 (Solicitação de Vaga) pode ser iniciada e puxada para o Doing.

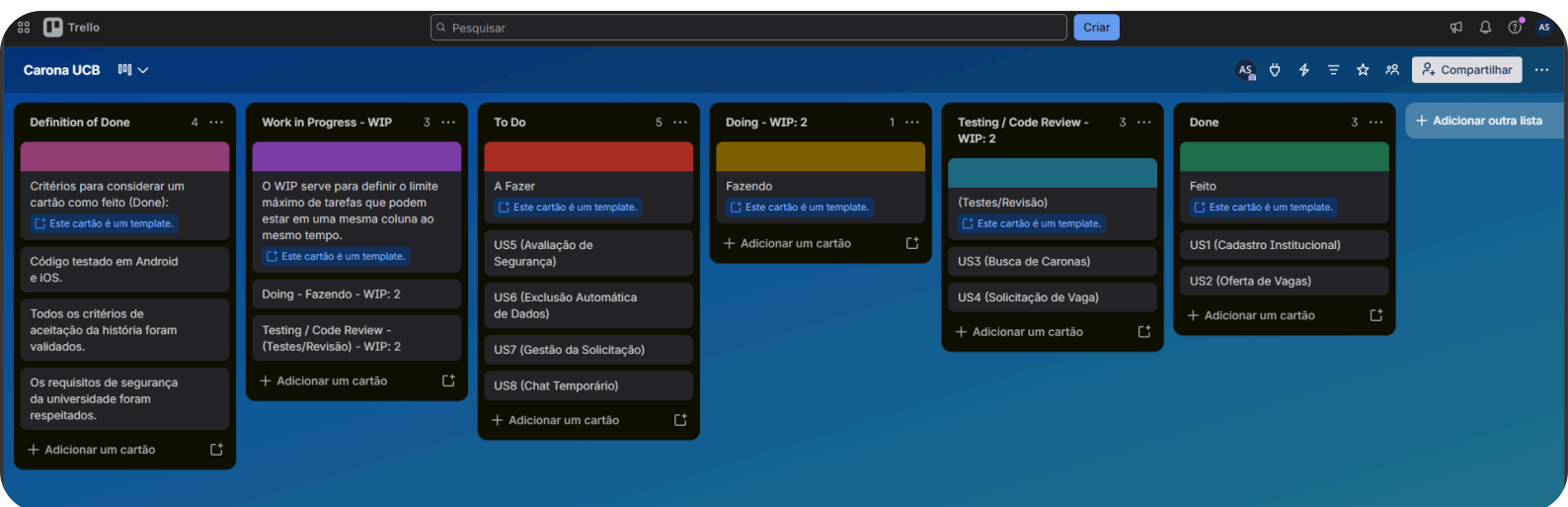
Dias 11 a 14:



Dias 11 a 14 da Sprint - Print Trello

A US3 foi movida para Testing / Code Review, atingindo o WIP máximo para a coluna de Testing/Code Review, uma vez que a equipe não conseguiu mover o US2 para a coluna Done, já que enfrentou problemas severos para validar o US 2.

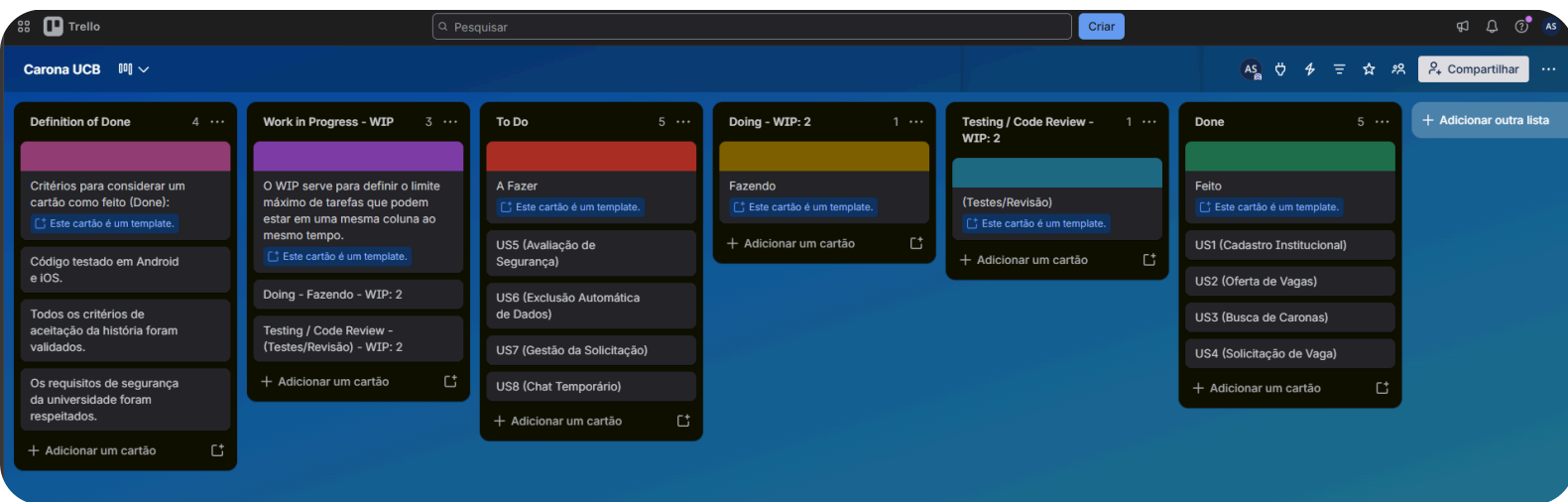
Dias 15 a 17:



Dias 15 a 17 da Sprint - Print Trello

A equipe conseguiu solucionar o gargalo da US2 movendo-a para a coluna Done e possibilitando que a US4 fosse movida para a coluna de Testing / Code Review.

Dias 18 a 20:



Dias 18 a 20 da Sprint - Print Trello

A equipe fez a validação da US3 e US4 movendo-as para a coluna Done finalizando a Sprint.

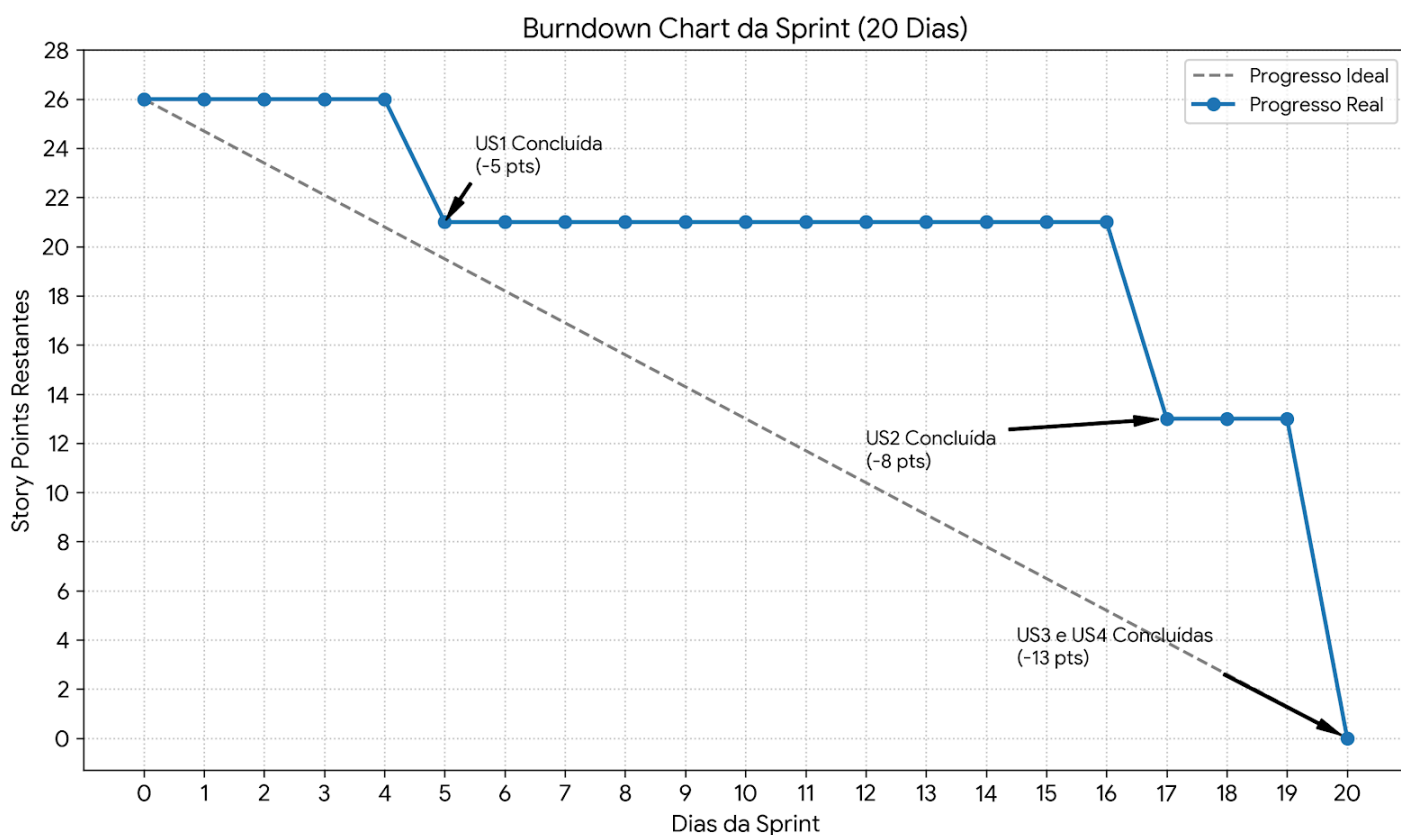
Bloqueios, Métricas e Aprendizados

Entre o décimo primeiro e o décimo quarto dia de execução, a equipe enfrentou um bloqueio na coluna *Testing / Code Review*, o motivo foi que a US2 (Oferta de Vagas) apresentou problemas complexos durante a etapa de validação, além disso o setor de TI da UCB também atrasou no fornecimento de informações cruciais para a conclusão e validação da US2.

Desse modo a US2 travou a coluna de testes junto com a US3, que havia acabado de ser concluída no desenvolvimento. Como o limite de WIP para a fase de testes estava definido em duas tarefas, a equipe atingiu a capacidade máxima e ficou impossibilitada de mover a US4 do *Doing* para a revisão. Diante disso, o time foi forçado a parar e concentrar todos os esforços para solucionar o gargalo da US2, conseguindo concluir sua validação e movê-la para a coluna *Done* apenas no décimo sétimo dia, o que finalmente liberou espaço no quadro para que o fluxo continuasse.

Em relação às métricas de acompanhamento do trabalho, o *Burndown Chart* iniciou com a projeção total de 26 Story Points. No quinto dia, a linha teve sua primeira queda para 21 pontos no momento em que a US1 atingiu a coluna *Done*. Após a entrega da US1 até a entrega da US2, o gráfico formou uma extensa linha reta horizontal, demonstrando a

ausência de entregas finalizadas. Contudo, após o destravamento no décimo sétimo dia, o gráfico observou uma nova queda, sendo zerado em apenas três dias após a conclusão da US2, demonstrando que o gargalo estava represando as entregas e atrapalhando o fluxo natural do trabalho.



Burndown chart da Sprint - Gráfico gerado pelo Gemini

Com esses acontecimentos a equipe amadureceu e conseguiu absorver aprendizados importantes para as próximas Sprints. O primeiro ensinamento foi a necessidade de aprimorar a estimativa técnica voltada para a garantia de qualidade, já que a equipe notou que a validação das regras de negócio, como a criação de ofertas de carona (US2), podem esconder desafios e bugs imprevistos; assim, ficou decidido que os cenários de teste deverão ser mapeados detalhadamente logo no planejamento. O segundo aprendizado foi sobre o poder do limite visual no Kanban, onde a restrição de no máximo duas tarefas simultâneas na coluna *Testing / Code Review* mostrou-se fundamental, pois, sem essa regra, o time teria continuado a desenvolver a US4 e a acumularia na coluna de testes junto com as outras, mascarando o problema da US2 e criando um gargalo ainda maior que muito provavelmente teria impedido a conclusão da Sprint.

Em síntese, a simulação desta primeira Sprint do CaronaUCB comprovou que as metodologias ágeis não apenas ajudam na entrega final do software, como também

auxiliam na capacidade de adaptação da equipe diante dos obstáculos. A aplicação rigorosa do Kanban, o respeito aos limites de WIP e a observação dos critérios para considerar uma US como feita (Dod), transformaram problemas técnicos em gargalos visíveis, forçando a colaboração e a tomada de decisão rápida. A equipe encerrou a sprint não apenas com o fluxo principal do aplicativo validado, mas com um time mais maduro, consciente das métricas e preparado para aplicar os ensinamentos obtidos na sprint nas próximas iterações do projeto.



Métodos Ágeis

CARONA UCB

Pedro Augusto Ferreira